

FICHE DE DÉMONSTRATION TECHNIQUE : INTÉGRATION BPMN

Traçabilité du fichier XML du Code Java jusqu'aux Tables PostgreSQL

Projet : IncidentFlow V2
Auteur : Anas Benhammou
Support de Démonstration Enseignant /
Jury

VOLET 1 : Comment le fichier XML est intégré dans le Backend (Le Code Java)

Spring Boot & Flowable
Engine

Le traitement du fichier XML s'articule en trois étapes précises dans le code source :

Étape A L'Ingestion du XML par Flowable (WorkflowController.java : ligne 22)

```
@PostMapping("/deploy-flowable")
public ResponseEntity<?> deployFlowableProcess(@RequestParam("file") MultipartFile file) {
    // 1. On donne le fichier XML brut au moteur Flowable
    Deployment deployment = repositoryService.createDeployment()
        .addInputStream("incident_workflow_v2.bpmn20.xml", file.getInputStream())
        .name("Dynamic Deployment from UI")
        .deploy(); // <-- Flowable valide le XML, incrémente la version et persiste en BDD

    // 2. On synchronise immédiatement notre application avec le nouveau modèle
    workflowService.syncWithFlowable(repositoryService);
    return ResponseEntity.ok("Déploiement réussi");
}
```

Ce que vous dites au professeur :

« Monsieur, dès qu'on envoie le fichier XML, c'est le service natif repositoryService de Flowable qui prend le relais. Il parse la structure XML, valide la syntaxe BPMN 2.0, incrémente la version du processus et stocke le binaire en base. »

Étape B La Lecture Dynamique du BPMN en Mémoire (WorkflowService.java : ligne 30)

```
// 1. On demande à Flowable l'arbre en mémoire du dernier BPMN déployé
BpmnModel bpmnModel = repositoryService.getBpmnModel(procDef.getId());

// 2. On parcourt les flèches (SequenceFlow) pour extraire les tâches dans l'ordre chronologique
List<UserTask> tasks = process.findFlowElementsOfType(UserTask.class);

// 3. On met à jour nos tables d'états pour alimenter le Kanban et le Stepper
workflow.getStates().clear();
for (String stateName : orderedStateNames) {
    workflow.getStates().add(new WorkflowState(stateName, ...));
}
workflowRepository.save(workflow);
```

Étape C L'Avancement d'un Incident lors d'une transition Kanban (IncidentService.java : ligne 208)

```
// Quand on déplace une carte sur le Kanban :
taskService.complete(currentTask.getId(), variables);

// Flowable calcule automatiquement la tâche suivante selon le XML BPMN :
Task nextTask = taskService.createTaskQuery().processInstanceId(incident.getProcessInstanceId()).singleResult();
incident.setStatus(nextTask.getName()); // <-- Le statut en BDD prend le nom littéral défini dans le XML
```

VOLET 2 : Où on voit ces modifications dans la Base de Données (Les Tables)

PostgreSQL / Schémas flowable &
incidentflow

Ouvrez **pgAdmin** devant l'enseignant ou exécutez ces 5 requêtes dans l'outil Query Tool pour prouver la persistance réelle :

1. La preuve que le XML a été déployé : `flowable.act_re_deployment`

```
SELECT id_, name_, deploy_time_  
FROM flowable.act_re_deployment  
ORDER BY deploy_time_ DESC LIMIT 1;
```

👉 **Ce qu'on voit** : Une nouvelle ligne apparaît avec l'identifiant unique du déploiement ainsi que la date et l'heure exacte de l'import du fichier.

2. La preuve physique que le fichier XML est en base : `flowable.act_ge_bytearray`

```
SELECT id_, name_, length(bytes_) AS "Taille (Octets)"  
FROM flowable.act_ge_bytearray  
WHERE name_ LIKE '%.bpmn%';
```

👉 **Ce qu'on voit** : Le fichier XML `incident_workflow_v2.bpmn20.xml` stocké sous forme binaire (BLOB/bytea). C'est la preuve physique que le XML n'est pas perdu dans la nature mais persisté de façon pérenne dans PostgreSQL.

3. La preuve du Versioning du Processus : `flowable.act_re_procdef`

```
SELECT id_, key_, version_, name_  
FROM flowable.act_re_procdef  
ORDER BY version_ DESC;
```

👉 **Ce qu'on voit** : La colonne `version_` qui s'incrémente (ex: 1, puis 2, puis 3).
Ce que vous dites au prof : « Vous voyez Monsieur, Flowable a créé une nouvelle version sans écraser l'ancienne, ce qui protège les incidents en cours. »

4. La preuve de la synchronisation Métier : `incidentflow.workflow_states`

```
SELECT id, name, label  
FROM incidentflow.workflow_states;
```

👉 **Ce qu'on voit** : Les nouveaux noms d'états extraits du XML apparaissent ici (ex: *Nouveau*, *Assigné*, *En cours*, *Résolu*, *Clôturé*). C'est cette table qui alimente le stepper de la fiche d'incident et les colonnes du Kanban !

5. La preuve de l'Audit Trail lors des transitions : `flowable.act_hi_taskinst`

```
SELECT name_, start_time_, end_time_, duration_  
FROM flowable.act_hi_taskinst  
ORDER BY start_time_ DESC LIMIT 5;
```

👉 **Ce qu'on voit** : Le journal d'audit immuable de Flowable qui consigne chaque passage d'état avec l'opérateur et la durée exacte passée sur chaque tâche (conformité SLA / ISO 27001).

💬 La phrase de synthèse à prononcer devant le professeur :

« Pour résumer Monsieur :

1. Côté Code : le fichier XML entre par `WorkflowController`, est validé par le moteur `RepositoryService` de Flowable, et notre méthode `syncWithFlowable` propage les nouvelles étapes vers le Frontend.

2. Côté Base de données : le binaire XML est sauvegardé dans `flowable.act_ge_bytearray`, sa définition incrémente la table `flowable.act_re_procdef`, et les états métiers sont reflétés dans `incidentflow.workflow_states` sans jamais mélanger les données grâce à nos 4 schémas partitionnés. »